

Bab 4

Pemrograman Shell Linux



4.1. Tujuan

Setelah mengikuti praktikum ini, mahasiswa diharapkan dapat:

- Memahami bagaimana membuat program yang berupa perintah-perintah shell Linux

4.2. Pemrograman Shell

Shell linux merupakan sekumpulan perintah-perintah yang digabungkan ke dalam sebuah file, namun pada perkembangannya, penggunaan shell linux lebih mendekati bahasa pemrograman yang sebenarnya, dimana pada pemrograman shell linux dikenal juga konsep iterasi, kondisional dan fungsi.

Skrip shell disimpan dalam modus teks biasa yang dapat ditulis dengan menggunakan editor apapun seperti vi, emacs, joe dan sebagainya. Nama file yang digunakan bebas, yang terpenting file tersebut harus diberi izin akses eksekusi.

Berikut contoh pembuatan skrip shell

```
$cat skripshellku
:
echo ini skrip shell buatanku
$
```

sebelum file skrip dapat dijalankan, file tersebut harus memiliki izin eksekusi terlebih dahulu. Berikut contoh untuk menset file skrip diatas agar dapat dijalankan \$chmod u+x skripshellku setelah itu skrip shell tersebut baru dapat dieksekusi:

```
$skripshellku
ini skrip shell buatanku
$
```

jika ternyata muncul pesan

```
skripshellku: cannot execute
```

ini berarti direktori tempat kerja kita belum tercantum pada search-path, untuk itu perlu diset terlebih dahulu dengan:

```
PATH=$PATH:
```

4.3 Konsep Pemrograman

4.3.1 Variabel

Variabel harus memiliki nama dengan aturan:

- boleh menggunakan angka
- boleh menggunakan huruf
- boleh menggunakan garis bawah

contoh:

```
var1
var_1
```

4.3.2. Memasukan nilai pada variabel

Memasukan nilai ke variabel dapat dilakukan dengan cara

```
variabel=5
```

Yang perlu diperhatikan adalah tidak diperbolehkan menggunakan spasi di sebelah kanan dan kiri tanda samadengan '='

untuk memasukan nilai string, dapat dilakukan dengan cara

```
nama=tolib
nama="joko tarub"
```

4.3.3 Mengakses Variabel

untuk mengakses nilai dari variabel digunakan tanda \$ di depan nama variabel yang digunakan

```
echo $nama
```

4.3.4. Parameter Posisi

paramater yang dimasukan sebagai argumen dari file skrip yang dijalankan dimasukkan ke variabel 1, 2, 3 dan seterusnya, sehingga untuk mengakses parameter tersebut digunakan pemanggilan variabel 1, 2 dan seterusnya, seperti pada contoh berikut:

```
$cat makan
:
#
echo "$1" sedang makan "$2"
$
```

jika dijalankan akan menghasilkan

```
$makan joko nasi
joko sedang makan nasi
```

Berikut beberapa variabel yang berkaitan dengan penggunaan parameter.

- \$# : Menyimpan jumlah argumen yang diberikan pada program
- \$? : Menyimpan nilai exit dari perintah yang dieksekusi sebelumnya
- \$0 : Menyimpan nama program shell
- \$* : Menyimpan semua argumen yg diberikan (\$1, \$2 dst)
- "\$@" : Menyimpan semua argumen dan dimasukkan dalam tanda petik ("\$1". "\$2" ...)

8.3.5. Tanda Petik dan Back-slash

Tanda Petik Ganda (“)

Digunakan untuk mengabaikan karakter spasi yang berada di antara karakter atau string, sehingga dapat menyimpan kalimat yang lebih dari satu kata pada sebuah variabel.

```
kal="hallo pren.."
```

disamping itu, shell juga akan tetap menerjemahkan karakter khusus yang digunakan dalam tanda petik ganda seperti pada karakter dolar.

```
$cat hello
:
kal="hello $LOGNAME"
echo $kal
$
```

jika dijalankan maka akan menampilkan nama user yang melakukan login. Jika nama login yang digunakan aftar, maka skrip diatas akan menghasilkan

```
Hello aftar
```

Tanda Petik Tunggal (')

Karakter petik tunggal hampir sama dengan karakter petik ganda, hanya saja shell akan mengabaikan karakter khusus yang ada di dalam tanda petik

```
$cat hello
:
kal='hello $LOGNAME'
echo $kal
$
```

jika dijalankan maka akan menampilkan

```
Hello $LOGNAME
```

Tanda Petik Terbalik (`)

Karakter petik tunggal terbalik digunakan jika akan mengisi sebuah variabel dengan hasil proses dari suatu perintah

```
$cat list
```

```
:
echo `ls`
$
```

jika skrip tersebut dijalankan akan menampilkan daftar semua file pada direktori tersebut.

```
$list
hello      makan      skripshellku
```

BackSlash (\)

Karakter backslash digunakan untuk mengabaikan evaluasi terhadap karakter khusus. Namun karakter backslash hanya dapat menyembunyikan satu karakter saja yang terletak persis di belakang backslash.

```
kal=hello\ john
```

dengan cara diatas, kalimat “hello john” akan dimasukan ke dalam variabel kal.

4.3.6. Membaca Masukan dari Keyboard

Untuk meminta masukan dari user digunakan cara sebagai berikut

```
read <variabel>
```

sebagai contoh

```
$cat umur
:
echo -n "Berapa umurmu : "
read umur
echo "Usia kamu = $umur"
$
```

Jika dijalankan maka akan diminta untuk memasukan umur kamu

```
$umur
Berapa umurmu : 20
Usia kamu = 20
```

4.3.7. Perintah Test

digunakan untuk mentest kondisi pada pernyataan kondisional dan iterasi. Ada 4 (empat) jenis operator yang digunakan yaitu operator integer, string, file dan logika. Bentuk perintah test ini adalah

```
test <ekspresi>
```

jika perintah ini menghasilkan nilai nol berarti test yang dilakukan bernilai benar, jika tidak nol berarti tidak benar. Hasil dari operasi test tersimpan dlam variabel \$?.

Operator Integer

Digunakan untuk membandingkan dua nilai integer

- `bil1 -eq bil2` Benar jika bil1 sama dengan bil2
- `bil1 -ne bil2` Benar jika bil1 tidak sama dengan bil2
- `bil1 -lt bil2` Benar jika bil1 lebih kecil dari bil2
- `bil1 -le bil2` Benar jika bil1 lebih kecil atau sama dengan bil2
- `bil1 -gt bil2` Benar jika bil1 lebih besar dari bil2
- `bil1 -ge bil2` Benar jika bil1 lebih besar atau sama dengan bil2

Operator String

Digunakan untuk membandingkan dua buah string

- `str1 = str2` Benar jika str1 sama dengan str2
- `str1 != str2` Benar jika str1 tidak sama dengan str2
- `str` Benar jika str bukan null
- `-n str` Benar jika panjang str lebih dari null
- `-z str` Benar jika str sama dengan nol

Operator untuk File

Digunakan untuk mengetahui keberadaan file, mengetahui jenis file dan sebagainya.

- `-d namafile` Benar jika namafile adalah sebuah direktori
- `-f namafile` Benar jika namafile adalah sebuah file biasa
- `-r namafile` Benar jika namafile adalah file yang dapat dibaca
- `-s namafile` Benar jika namafiel panjangnya lebih dari nol
- `-w namafile` Benar jika namafiel adalah file yang dapat ditulis
- `-x namafile` Benar jika namafiel adalah file yang dapat dieksekusi

Operator Logika

Digunakan untuk mengetahui keberadaan file, mengetahui jenis file dan sebagainya.

- `expr` Benar jika expr tidak bernilai benar
- `expr1 -a expr2` Benar jika expr1 dan expr2 bernilai benar
- `expr1 -o expr2` Benar jika expr1 atau expr2 bernilai benar

8.3..8. Pernyataan Kondisional

bash shell memiliki pernyataan kondisional if dan case

Pernyataan IF

Bentuk dari pernyataan kondisional-if ada beberapa macam. Bentuk pertamanya yaitu:

```
if <eskpresi>
then
    <perintah>
fi
```

Bentuk kedua yaitu:

```
if <ekspresi>
```

```
then
    <perintah1>
else
    <perintah2>
fi
```

Bentuk ketiga yaitu:

```
if <ekspresi>
then
    <perintah1>
elif <ekspresi2>
then
    <perintah2>
else
    <perintah3>
fi
```

Pernyataan Case

fungsi pernyataan case sama seperti pernyataan if untuk memeriksa suatu ekspresi dan melakukan suatu proses bila ekspresi tersebut benar. Kelebihan dari case yaitu dapat menggunakan wildcards dalam ekspresi yang diperiksa.

```
case string1 in
<str1>)
    <perintah1>
<str2>)
    <perintah2>
*)
    <perintah3>
esac
```

4.3.8. Pernyataan Perulangan

Pernyataan for

pernyataan for digunakan untuk melakukan perulangan yang jumlah perulangannya sudah ditentukan sebelumnya. Bentuk pertama dari pernyataan for adalah

```
for var in <list>
do
    <perintah>
done
```

list berupa variabel yang berisi beberapa kata atau beberapa nilai yang sengaja dimasukan, di setiap kali perulangan variabel var akan diisi oleh tiap item yang ada di dalam list.

Bentuk kedua akan melakukan satu kali iterasi untuk tiap item yang ada di variabel var, dimana variabel var berisi list dari semua parameter yang di berikan pada command line:

```
for var
do
    <perintah>
done
```

pernyataan for di atas ekuivalen dengan pernyataan for berikut

```
vor var in "$@"
do
    <perintah>
done
```

Pernyataan While

Pernyataan while digunakan untuk melakukan perulangan dengan kondisi tertentu. Perulangan akan terus dilakukan selama ekspresi masih dipenuhi.

```
while <ekspresi>
do
    <perintah>
done
```

Pernyataan Until

Pernyataan until memiliki kemiripan dengan while, hanya saja pernyataan until akan melakukan perulangan saat kondisi ekspresi bernilai salah dan berhenti saat ekspresi bernilai benar.

```
until <ekspresi>
do
    <perintah>
done
```

Pernyataan Shift

Perintah shift ini digunakan untuk menggeser parameter satu posisi ke kiri, sehingga jika isi parameter seperti berikut

```
$1 = parameter1,    $2 = parameter2,    $3 = parameter3
```

maka perintah shift akan menjadikan isi parameter di atas berubah menjadi seperti berikut

```
$1 = parameter2,    $2 = parameter3
```

penggeseran dapat juga dilakukan lebih dari satu posisi dengan menambahkan angka setelah perintah shift

```
shift <jumlah posisi geser>
```

4.3.9. Fungsi

fungsi digunakan untuk mengelompokkan rutin yang dapat digunakan kembali selanjutnya. Untuk membuat fungsi digunakan cara berikut:

```
<nama fungsi> () {  
    <perintah>  
}
```

sedangkan untuk melakukan pemanggilan fungsi dapat dilakukan dengan cara berikut

```
<nama fungsi> [parameter1 parameter2 .....]
```